

**SiForLPG**

Forecasting ARIMA · SARIMA

DOKUMENTASI TEKNIS SISTEM

Sistem Informasi Forecasting Permintaan LPG

Dokumentasi lengkap arsitektur, struktur folder, fungsi per-modul, alur kerja engine ARIMA & SARIMA, skema basis data, serta lapisan antarmuka dari aplikasi **SiForLPG** — dibangun dengan Flask, Supabase (PostgreSQL), dan pustaka time series Python.

STUDI KASUS

PT Pertamina Patra Niaga
Pangkalan Susu, Kab. Langkat

STACK TEKNOLOGI

Flask · Supabase · pmdarima
statsmodels · Pandas

VERSI DOKUMEN

1.0 — Juni 2026

Daftar Isi

Struktur pembahasan dokumentasi teknis sistem SiForLPG

1	Pendahuluan
1.1	Deskripsi Sistem
1.2	Fitur Utama
1.3	Tumpukan Teknologi (Tech Stack)
2	Arsitektur & Struktur Proyek
2.1	Pola Arsitektur (Application Factory + Blueprint)
2.2	Struktur Folder Lengkap
3	Konfigurasi & Entry Point
3.1	run.py — Entry Point Aplikasi
3.2	config.py — Konfigurasi Aplikasi
3.3	app/__init__.py — Application Factory
3.4	app/supabase_client.py — Koneksi Database
4	Lapisan Model (app/models)
4.1	admin.py — Autentikasi & Akun Admin
4.2	data_lpg.py — Operasi Data Penyaluran LPG
4.3	forecast_run.py — Operasi Hasil Forecasting
4.4	import_log.py & export_log.py — Log Aktivitas
5	Lapisan Route / Blueprint (app/routes)
5.1	auth.py — Autentikasi
5.2	dashboard.py — Dashboard Utama
5.3	data_management.py — Kelola Dataset
5.4	forecasting.py — Proses Forecasting
5.5	api.py — Endpoint AJAX
6	Lapisan Service (app/services)
6.1	excel_import.py — Import & Validasi Dataset
6.2	forecast_engine.py — Engine ARIMA & SARIMA
6.3	excel_export.py — Generator Laporan Excel
7	Utilitas (app/utils)
8	Skema Basis Data (Supabase / PostgreSQL)
9	Lapisan Antarmuka (Frontend & Desain Sistem)
9.1	Struktur Template (Jinja2)
9.2	Sistem Desain Neumorphism
9.3	JavaScript & Interaktivitas
10	Alur Kerja Sistem End-to-End

11 Keamanan & Catatan Produksi

12 Troubleshooting

Pendahuluan

1.1 Deskripsi Sistem

SiForLPG adalah sistem informasi berbasis web yang dibangun menggunakan framework **Flask (Python)** untuk meramalkan (forecasting) permintaan penyaluran LPG harian menggunakan algoritma **ARIMA** dan **SARIMA**. Sistem ini dikembangkan untuk mendukung penelitian skripsi "Forecasting Permintaan LPG Menggunakan Algoritma ARIMA dan SARIMA Berdasarkan Data Historis Penyaluran", dengan studi kasus PT Pertamina Patra Niaga, wilayah Pangkalan Susu, Kabupaten Langkat, Sumatera Utara.

Seluruh data aplikasi disimpan pada **Supabase** (layanan PostgreSQL terkelola), sementara proses komputasi time series (uji stasioneritas, differencing, ACF/PACF, auto-order ARIMA/SARIMA, evaluasi model) dijalankan langsung di sisi server Flask menggunakan **statsmodels** dan **pmdarima**.

1.2 Fitur Utama

Autentikasi Admin

LOGIN MANUAL, BCrypt
HASH

Import Excel

DETEKSI SHEET OTOMATIS

ARIMA & SARIMA

AUTO-ORDER VIA PMDARIMA

Export Excel

LAPORAN 11 SHEET

- **Kelola Data** — melihat, memfilter (per kabupaten/kota & rentang tanggal), dan menghapus dataset.
- **Proses Forecasting Lengkap** — agregasi harian, split train/test, uji ADF, differencing otomatis, analisis ACF/PACF, pemodelan ARIMA & SARIMA (auto-order), evaluasi MAE/RMSE/MAPE, peramalan ke depan, serta perbandingan fleksibel dengan data aktual 2026.
- **Visualisasi Detail** — setiap tahap proses ditampilkan lengkap (grafik train/test, korelogram ACF/PACF, prediksi vs aktual).
- **Tema Light / Dark / System** tersimpan di **localStorage**.
- **Desain Neumorphism** dengan palet warna Pinky · Sky Blue · Teeny Greeny.

1.3 Tumpukan Teknologi (Tech Stack)

Lapisan	Teknologi	Keterangan
Backend Framework	Flask (Python)	Application Factory pattern + Blueprint modular per domain
Basis Data	Supabase (PostgreSQL)	Diakses melalui supabase-py client (anon key), bukan ORM
Autentikasi	Flask Session + bcrypt	Login manual (bukan Supabase Auth), cookie HttpOnly , masa berlaku 8 jam
Time Series Engine	statsmodels, pmdarima	ADF test, ACF/PACF, ARIMA, SARIMAX, auto_arima (stepwise AIC search)
Data Processing	pandas, numpy	Agregasi harian, reindexing tanggal, perhitungan evaluasi model
Evaluasi Model	scikit-learn	mean_absolute_error, mean_squared_error (dasar MAE/RMSE/MAPE)
Import/Export Excel	pandas, openpyxl	Baca sheet dataset & tulis laporan multi-sheet bergaya
Frontend	Jinja2, CSS murni (neumorphism), Chart.js	Tanpa framework JS besar; theme switcher custom via localStorage
Font & Ikon	Plus Jakarta Sans, Inter, Font Awesome 6	Dimuat via CDN cdnjs.cloudflare.com

Arsitektur & Struktur Proyek

2.1 Pola Arsitektur (Application Factory + Blueprint)

Aplikasi memakai pola **Application Factory** Flask (fungsi `create_app()`) yang membuat instance aplikasi, mendaftarkan **5 Blueprint** modular per domain (auth, dashboard, data, forecast, api), serta mendaftarkan error handler global. Setiap Blueprint memiliki tanggung jawab tunggal (single responsibility) dan saling terhubung melalui pemanggilan fungsi pada lapisan `app/models` dan `app/services`.

Arsitektur sistem terbagi menjadi 4 lapisan utama:

- 1 Routes (app/routes)**
Menangani HTTP request/response, validasi input form, memanggil model & service, merender template.
- 2 Services (app/services)**
Logika bisnis inti — pembacaan/validasi Excel, seluruh pipeline forecasting ARIMA/SARIMA, dan pembuatan laporan Excel.
- 3 Models (app/models)**
Lapisan akses data — seluruh query/insert/update ke Supabase dikelompokkan per tabel/domain.
- 4 Templates & Static (app/templates, app/static)**
Lapisan presentasi — Jinja2 template bertema neumorphism dengan JS minimal (Chart.js & theme switcher).

2.2 Struktur Folder Lengkap

```
lpg_forecast/
├── app/
│   ├── models/ — lapisan akses data Supabase
│   │   ├── admin.py
│   │   ├── data_lpg.py
│   │   ├── export_log.py
│   │   ├── forecast_run.py
│   │   └── import_log.py
│   ├── routes/ — blueprint HTTP
│   │   ├── api.py
│   │   ├── auth.py
│   │   ├── dashboard.py
│   │   ├── data_management.py
│   │   └── forecasting.py
│   ├── services/ — logika bisnis inti
│   │   ├── excel_export.py
│   │   ├── excel_import.py
│   │   └── forecast_engine.py
│   ├── static/
│   │   ├── css/style.css — design system neumorphism
│   │   ├── img/
│   │   └── js/theme.js — theme switcher & util UI
│   ├── templates/
│   │   ├── admin/ — 8 halaman admin
│   │   ├── auth/ — login, profil
│   │   ├── errors/ — 404, 413, 500
│   │   ├── partials/ — flash, navbar, sidebar
│   │   └── base.html — layout induk
│   ├── utils/
│   │   └── decorators.py — @login_required
│   ├── __init__.py — application factory
│   └── supabase_client.py — singleton koneksi DB
├── sql/
│   ├── schema.sql — 12 tabel + index + trigger
│   └── seed.sql — akun admin default + master wilayah
├── uploads/ — folder sementara file Excel yang diupload
├── config.py — konfigurasi terpusat (.env)
├── run.py — entry point: python run.py
├── requirements.txt
├── .env — kredensial Supabase (tidak di-commit)
└── README.md
```

Catatan: Tidak ada folder `controllers` terpisah dari `routes` — pola Flask Blueprint menggabungkan peran controller langsung pada modul di `app/routes/`, sehingga setiap file route setara dengan satu controller domain.

Konfigurasi & Entry Point

Empat berkas inti yang menyusun fondasi aplikasi sebelum lapisan route, service, dan model dijalankan.

run.py

ENTRY POINT

Titik masuk untuk menjalankan aplikasi melalui perintah `python run.py`. Memanggil `create_app()` dari `app/__init__.py` lalu menjalankan Flask development server pada `host 0.0.0.0`, `port 5000`, dengan mode `debug` mengikuti konfigurasi `Config.DEBUG`.

config.py

KONFIGURASI

Kelas `Config` membaca seluruh variabel lingkungan dari berkas `.env` menggunakan `python-dotenv`. Berisi pengaturan kunci sebagai berikut:

Konstanta	Nilai / Sumber	Kegunaan
<code>SECRET_KEY</code>	<code>env SECRET_KEY</code>	Kunci enkripsi session Flask
<code>SUPABASE_URL / SUPABASE_KEY</code>	<code>env</code>	Kredensial koneksi ke project Supabase
<code>MAX_CONTENT_LENGTH</code>	25 MB	Batas ukuran file upload (memicu error 413 jika melebihi)
<code>UPLOAD_FOLDER</code>	<code>lpg_forecast/uploads</code>	Lokasi penyimpanan sementara file Excel yang diupload
<code>ALLOWED_EXTENSIONS</code>	<code>{xlsx, xls}</code>	Validasi ekstensi file dataset
<code>SESSION_COOKIE_HTTPONLY / SAMESITE</code>	<code>True / Lax</code>	Pengamanan cookie session admin
<code>PERMANENT_SESSION_LIFETIME</code>	8 jam	Masa berlaku sesi login admin
<code>EXPECTED_COLUMNS</code>	3 kolom wajib	Validasi struktur kolom Excel saat import
<code>SHEET_HISTORIS_DEFAULT / SHEET_AKTUAL_DEFAULT</code>	nama sheet default	Nama sheet acuan sesuai proposal skripsi

app/__init__.py

APPLICATION FACTORY

Fungsi	Deskripsi
<code>create_app()</code>	Membuat instance Flask, memuat <code>Config</code> , membuat folder upload jika belum ada, mendaftarkan 5 Blueprint (<code>auth_bp</code> , <code>dashboard_bp</code> , <code>data_bp</code> , <code>forecast_bp</code> , <code>api_bp</code>), mendaftarkan context processor <code>inject_globals()</code> (menyuntikkan <code>current_year</code> , status login, dan nama admin ke seluruh template), serta mendaftarkan error handler untuk kode 404 , 413 , dan 500 .

Fungsi	Deskripsi
<code>get_supabase()</code>	Mengembalikan instance singleton Supabase client. Inisialisasi hanya terjadi sekali (lazy init dengan variabel global <code>_supabase_client</code>), memvalidasi bahwa <code>SUPABASE_URL</code> dan <code>SUPABASE_KEY</code> sudah diatur, lalu mengembalikan client yang sama di seluruh aplikasi agar tidak membuka koneksi berulang.

Lapisan Model (app/models)

Seluruh fungsi pada lapisan ini berkomunikasi langsung dengan Supabase melalui `get_supabase()`. Tidak menggunakan ORM — query ditulis langsung memakai chain method `supabase-py`.

admin.py

AUTENTIKASI & AKUN ADMIN

app/models/admin.py

Fungsi	Parameter	Deskripsi
<code>hash_password()</code>	<code>plain_password</code>	Hash password memakai bcrypt (gensalt otomatis)
<code>verify_password()</code>	<code>plain_password</code> , <code>password_hash</code>	Verifikasi password terhadap hash; aman terhadap <code>ValueError</code> / <code>AttributeError</code>
<code>get_admin_by_username()</code>	<code>username</code>	Ambil 1 baris admin berdasarkan username
<code>get_admin_by_id()</code>	<code>admin_id</code>	Ambil 1 baris admin berdasarkan UUID
<code>touch_last_login()</code>	<code>admin_id</code>	Update kolom <code>last_login_at</code> ke waktu UTC saat ini
<code>update_password()</code>	<code>admin_id</code> , <code>new_password</code>	Hash & simpan password baru
<code>update_profile()</code>	<code>admin_id</code> , <code>full_name</code> , <code>email</code>	Update sebagian (partial update) nama lengkap/email admin

app/models/data_lpg.py

Mengelola dua tabel data utama (`data_historis` & `data_aktual_2026`) melalui konstanta `TABLE_MAP` , serta tabel master `wilayah` dan ringkasan `kontribusi_wilayah` .

Fungsi	Parameter	Deskripsi
<code>_get_wilayah_map()</code>	—	Ambil seluruh master wilayah → dict <code>{nama: id}</code>
<code>_ensure_wilayah()</code>	<code>nama_list</code>	Memastikan setiap nama wilayah baru otomatis ter-insert ke tabel master <code>wilayah</code> (upsert), termasuk penandaan <code>is_top_kontributor</code> berdasarkan daftar default 5 wilayah
<code>insert_batch()</code>	<code>tipe, rows, batch_size=500</code>	Insert data secara batch (upsert dengan conflict key <code>tanggal, nama_wilayah, sumber_import</code>), mengisi <code>wilayah_id</code> dari hasil lookup, lalu memicu refresh kontribusi
<code>_refresh_kontribusi_wilayah()</code>	<code>tipe</code>	Menghitung ulang total & persentase kontribusi per wilayah memakai <code>pandas</code> , lalu menyimpannya ke tabel <code>kontribusi_wilayah</code>
<code>count_rows()</code>	<code>tipe</code>	Hitung jumlah baris pada tabel (count exact)
<code>get_date_range()</code>	<code>tipe</code>	Ambil tanggal minimum & maksimum pada tabel
<code>get_distinct_wilayah()</code>	<code>tipe</code>	Daftar nama wilayah unik (dengan pagination 1000 baris/halaman)
<code>fetch_aggregated_daily()</code>	<code>tipe, wilayah, tanggal_mulai, tanggal_akhir</code>	Ambil seluruh baris mentah (<code>tanggal, wilayah, total_berat</code>) dengan filter opsional & pagination, untuk diagregasi pada layer service
<code>delete_all()</code>	<code>tipe</code>	Hapus seluruh baris pada tabel dataset
<code>get_kontribusi_wilayah()</code>	<code>tipe='historis'</code>	Hitung kontribusi total & persentase per wilayah secara real-time (dipakai dashboard & halaman kontribusi)

app/models/forecast_run.py

Mengelola 1 tabel header (`forecast_run`) dan 4 tabel detail (`forecast_acf_pacf` , `forecast_dataset_point` , `forecast_prediction_test` , `forecast_prediction_future`).

Fungsi	Parameter	Deskripsi
<code>_clean_num()</code>	v	Konversi nilai NaN/Inf menjadi <code>None</code> agar aman disimpan sebagai JSON/Postgres numeric
<code>create_run()</code> / <code>update_run()</code>	payload	Insert/Update baris header <code>forecast_run</code>
<code>get_run()</code> / <code>list_runs()</code> / <code>delete_run()</code>	run_id / limit	Ambil 1 run, daftar run terurut terbaru, atau hapus run (cascade ke tabel detail)
<code>save_acf_pacf()</code>	run_id, tahap, items	Simpan nilai ACF/PACF per lag untuk tahap "sebelum_diff" / "sesudah_diff"
<code>save_dataset_points()</code>	run_id, train_series, test_series	Simpan snapshot data harian train & test (untuk grafik & export)
<code>save_prediction_test()</code>	run_id, model, test_series, pred_mean, pred_ci	Simpan hasil prediksi pada data test beserta residual & interval kepercayaan 95%
<code>save_prediction_future()</code>	run_id, model, future_mean, future_ci, actual_2026_map	Simpan hasil peramalan masa depan, termasuk nilai aktual 2026 jika tanggalnya cocok
<code>get_acf_pacf()</code> / <code>get_dataset_points()</code>	run_id	Ambil data ACF/PACF atau dataset point untuk ditampilkan di halaman detail
<code>get_prediction_test()</code> / <code>get_prediction_future()</code>	run_id, model	Ambil hasil prediksi test/future, opsional difilter per model (ARIMA/SARIMA)

Fungsi	Deskripsi
<code>create_log()</code>	Catat hasil 1 proses import (sukses/gagal/sebagian)
<code>list_logs()</code>	Daftar log import terbaru (default 20)

Fungsi	Deskripsi
<code>create_export_log()</code>	Catat setiap kali admin mengunduh laporan Excel hasil forecasting

Lapisan Route / Blueprint (app/routes)

5 Blueprint menangani seluruh endpoint HTTP aplikasi. Semua route admin (kecuali login) dilindungi decorator `@login_required`.

auth.py

url_prefix: /

app/routes/auth.py – Blueprint: auth_bp

Method	URL	View Function	Deskripsi
GET/POST	/login	<code>login()</code>	Form login admin; validasi username/password terhadap hash bcrypt, set session (<code>admin_id</code> , <code>admin_name</code> , <code>admin_username</code>), redirect ke <code>next</code> atau dashboard
GET	/logout	<code>logout()</code>	Hapus seluruh session & redirect ke login
GET/POST	/profil	<code>profile()</code>	Lihat & ubah profil admin. Aksi <code>update_profile</code> mengubah nama/email; aksi <code>change_password</code> memvalidasi password lama, panjang minimal 6 karakter, dan kecocokan konfirmasi

dashboard.py

url_prefix: /

app/routes/dashboard.py – Blueprint: dashboard_bp

Method	URL	View Function	Deskripsi
GET	/	<code>root()</code>	Redirect ke <code>/dashboard</code>
GET	/dashboard	<code>index()</code>	Mengumpulkan ringkasan: jumlah baris historis/aktual, rentang tanggal, top-5 kontribusi wilayah, 5 run forecasting terbaru, serta data grafik tren harian (label & nilai) untuk Chart.js

app/routes/data_management.py – Blueprint: data_bp

Method	URL	View Function	Deskripsi
GET	/data/	<code>index()</code>	Ringkasan jumlah baris, rentang tanggal kedua dataset, dan 10 log import terbaru
GET/POST	/data/upload	<code>upload()</code>	Form & proses upload Excel: simpan file sementara, deteksi/pilih sheet, jalankan <code>process_upload()</code> , <code>insert_batch()</code> , catat <code>import_log</code> , lalu hapus file fisik dari server
POST	/data/sheets	<code>preview_sheets()</code>	Endpoint AJAX: upload file sementara → kembalikan daftar nama sheet (JSON) untuk dipilih user sebelum import final
GET	/data/historis	<code>view_historis()</code>	Tampilkan data historis dengan filter wilayah/tanggal + grafik tren (delegasi ke <code>_view_dataset()</code>)
GET	/data/aktual-2026	<code>view_aktual()</code>	Tampilkan data aktual 2026 dengan filter serupa
–	(internal)	<code>_view_dataset()</code>	Helper bersama: ambil wilayah unik, ambil baris terfilter, agregasi harian via pandas untuk grafik, batasi tabel ke 500 baris pertama (terurut)
GET	/data/kontribusi	<code>kontribusi()</code>	Tampilkan kontribusi total & persentase per kabupaten/kota (parameter <code>tiipe</code>)
POST	/data/hapus/<tiipe>	<code>hapus()</code>	Hapus seluruh baris dataset historis atau aktual_2026

app/routes/forecasting.py – Blueprint: forecast_bp

Method	URL	View Function	Deskripsi
GET	/forecast/	<code>index()</code>	Daftar riwayat hingga 50 run forecasting terakhir
GET/POST	/forecast/baru	<code>new_run()</code>	Form input parameter (nama run, scope wilayah, rasio train/test, rentang tanggal prediksi, opsi bandingkan aktual) pada GET; pada POST menjalankan <code>run_full_pipeline()</code> , menghitung model terbaik, opsional membandingkan dengan data aktual 2026, lalu menyimpan seluruh hasil ke 5 tabel Supabase melalui <code>app.models.forecast_run</code>
GET	/forecast/<run_id>	<code>detail()</code>	Ambil & tampilkan seluruh detail satu run: dataset point, ACF/PACF sebelum & sesudah differencing, prediksi test & future untuk kedua model
POST	/forecast/<run_id>/hapus	<code>hapus_run()</code>	Hapus 1 run forecasting (cascade ke tabel detail terkait)
GET	/forecast/<run_id>/export	<code>export_excel()</code>	Bangun & unduh laporan Excel 11-sheet via <code>build_forecast_excel()</code> , catat ke <code>export_log</code>

Validasi pada `new_run()` : tanggal akhir prediksi tidak boleh sebelum tanggal mulai, dan horizon peramalan dibatasi maksimal **366 hari**.

app/routes/api.py – Blueprint: api_bp

Method	URL	View Function	Deskripsi
GET	/api/wilayah/<tipe>	wilayah()	Kembalikan JSON daftar wilayah unik untuk mengisi dropdown filter secara dinamis
GET	/api/series/<tipe>	series()	Kembalikan JSON <code>{labels, values}</code> deret waktu harian terfilter (wilayah, rentang tanggal) untuk dirender ulang oleh Chart.js tanpa reload halaman

Lapisan Service (app/services)

Berisi seluruh logika bisnis inti aplikasi: validasi & transformasi data Excel, mesin forecasting time series, dan penyusun laporan Excel multi-sheet.

6.1 excel_import.py — Import & Validasi Dataset

app/services/excel_import.py

Membaca & memvalidasi file Excel sesuai struktur `datapenelitian1.xlsx`, lalu menyiapkannya untuk disimpan ke Supabase. Kolom wajib: `Act.`, `Gds`, `Mvmnt Date`, `Kabupaten/Kota`, `Total Berat`.

Fungsi	Deskripsi
<code>list_sheets()</code>	Kembalikan daftar nama sheet pada file Excel
<code>read_sheet()</code>	Baca 1 sheet menjadi DataFrame pandas
<code>validate_columns()</code>	Pastikan 3 kolom wajib ada; lempar <code>ImportValidationError</code> jika tidak, sambil menyebutkan kolom yang ditemukan
<code>normalize_dataframe()</code>	Normalisasi: parsing tanggal (mendukung serial number Excel maupun format tanggal biasa, origin 1899-12-30), uppercase & trim nama wilayah, konversi numerik <code>total_berat</code> , lalu membuang baris dengan nilai kosong/tidak valid
<code>aggregate_duplicates()</code>	Jumlahkan baris dengan kombinasi (tanggal, wilayah, sumber_import) yang sama — penting karena constraint UNIQUE pada tabel Supabase & karena dataset granular bisa memiliki banyak baris kecil per hari
<code>dataframe_to_records()</code>	Konversi DataFrame final menjadi list of dict yang JSON-serializable (tipe numpy → tipe Python native)
<code>process_upload()</code>	Pipeline lengkap satu pintu: <code>read_sheet</code> → <code>normalize_dataframe</code> → <code>aggregate_duplicates</code> → <code>dataframe_to_records</code> , mengembalikan <code>(records, ringkasan)</code> berisi statistik baris mentah/valid/dibuang/ setelah agregasi serta jumlah wilayah

`class ImportValidationError(Exception)` — exception kustom yang ditangkap secara eksplisit di route `upload()` untuk menampilkan pesan error yang ramah pengguna (berbeda dari exception generik lainnya).

6.2 forecast_engine.py — Engine ARIMA & SARIMA

app/services/forecast_engine.py

Modul inti penelitian. Konstanta `SEASONAL_PERIOD = 7` merepresentasikan pola musiman mingguan untuk data penyaluran harian.

Fungsi-fungsi pendukung

Fungsi	Deskripsi
<code>build_daily_series()</code>	Agregasi <code>total_berat</code> per tanggal (sum seluruh baris pada scope terfilter), lalu reindex ke deret tanggal harian kontinu dan interpolasi linear + backfill/forward-fill untuk mengisi tanggal yang hilang
<code>train_test_split_series()</code>	Split deret waktu menjadi train/test berdasarkan <code>train_ratio</code> (default 0.8) secara berurutan (bukan acak) — sesuai praktik time series
<code>adf_test()</code>	Uji Augmented Dickey-Fuller (<code>statsmodels.adfuller</code> , autolag AIC); stasioner jika p-value < 0.05
<code>determine_differencing()</code>	Differencing otomatis berulang (maksimal <code>d=2</code>) sampai data dinyatakan stasioner oleh ADF test; mengembalikan order d serta hasil ADF sebelum & sesudah
<code>compute_acf_pacf()</code>	Hitung ACF (FFT) & PACF (metode YWM) hingga maksimal 30 lag, dibatasi oleh panjang data

<code>evaluate_predictions()</code>	Hitung MAE, RMSE (akar dari MSE), dan MAPE (menghindari pembagian dengan nol menggunakan mask nilai aktual $\neq 0$)
<code>fit_arima_auto()</code>	Cari order (p,d,q) terbaik non-seasonal via <code>pmdarima.auto_arima</code> (stepwise, max_p=5, max_q=5, max_d=2), lalu fit ulang dengan <code>statsmodels.ARIMA</code>
<code>fit_sarima_auto()</code>	Cari order (p,d,q)(P,D,Q)s terbaik via <code>auto_arima</code> seasonal=True (m=7), lalu fit dengan <code>SARIMAX</code> (enforce_stationarity/invertibility=False untuk stabilitas numerik)
<code>predict_test_arima()</code> / <code>predict_test_sarima()</code>	Forecast sepanjang data test, mengembalikan nilai prediksi rata-rata & interval kepercayaan 95%
<code>refit_full_and_forecast_future()</code>	Melatih ulang model dengan seluruh data (train+test) memakai order yang sudah ditemukan, lalu meramalkan ke tanggal-tanggal masa depan pilihan admin
<code>determine_best_model()</code>	Sistem voting 3 metrik (MAE, RMSE, MAPE) pada data test; model dengan nilai lebih kecil pada tiap metrik mendapat 1 poin; dasi diputuskan oleh RMSE terendah
<code>run_full_pipeline()</code>	Orkestrator utama yang memanggil seluruh fungsi di atas secara berurutan dan mengembalikan dict besar berisi semua hasil intermediate

Alur Pipeline `run_full_pipeline()`

- 1 Agregasi Harian**
Baris mentah hasil filter scope wilayah diagregasi menjadi satu deret waktu harian kontinu (`build_daily_series`).
- 2 Split Train/Test**
Deret dipecah berurutan sesuai `train_ratio` yang dipilih admin.
- 3 Uji Stasioneritas (ADF)**
Dijalankan pada data train sebelum differencing.
- 4 Differencing Otomatis**
Bila belum stasioner, differencing diulang hingga maksimal d=2 atau hingga stasioner.
- 5 Analisis ACF & PACF**
Dihitung sebelum & (bila perlu) sesudah differencing sebagai bahan korelogram.
- 6 Pemodelan ARIMA**
Auto-order non-seasonal via `pmdarima`, fit pada data train, evaluasi pada data test.
- 7 Pemodelan SARIMA**
Auto-order seasonal (periode 7 hari) via `pmdarima`, fit pada data train, evaluasi pada data test.
- 8 Evaluasi & Perbandingan**
MAE, RMSE, MAPE dihitung untuk kedua model pada data test; model terbaik ditentukan via sistem voting.
- 9 Peramalan Masa Depan**
Kedua model di-refit dengan seluruh data (train+test), lalu meramalkan sesuai rentang tanggal yang dipilih admin (maks. 366 hari).
- 10 Perbandingan Aktual 2026 (opsional)**
Jika diaktifkan & tanggal forecast tumpang-tindih dengan data aktual 2026, dihitung evaluasi tambahan MAE/RMSE/MAPE terhadap data riil.

6.3 excel_export.py — Generator Laporan Excel

`app/services/excel_export.py`

Membangun laporan Excel multi-sheet menggunakan `openpyxl` dengan styling konsisten (header biru `#2A6EAF` sesuai warna brand, judul bold, border tipis, auto-size kolom). Fungsi utama `build_forecast_excel()` menyusun **11 sheet** berikut:

#	Nama Sheet	Isi
---	------------	-----

1	1_Ringkasan	Info run, periode data & prediksi, proporsi train/test, model terbaik beserta skor
2	2_Data_Harian	Seluruh titik data harian (train & test) yang dipakai dalam run
3	3_Uji_Stasioneritas	Hasil ADF test sebelum & sesudah differencing
4	4_ACF_PACF	Nilai ACF/PACF per lag, sebelum & sesudah differencing
5	5_Parameter_ARIMA	Order (p,d,q), AIC, BIC, MAE/RMSE/MAPE (test & vs aktual 2026)
6	6_Parameter_SARIMA	Order (p,d,q)(P,D,Q,s), AIC, BIC, MAE/RMSE/MAPE (test & vs aktual 2026)
7	7_Prediksi_ARIMA_Test	Aktual vs prediksi ARIMA, residual, batas bawah/atas interval 95%
8	8_Prediksi_SARIMA_Test	Aktual vs prediksi SARIMA, residual, batas bawah/atas interval 95%
9	9_Peramalan_vs_Aktual2026	Prediksi ARIMA & SARIMA masa depan digabung (merge outer per tanggal) beserta aktual 2026 bila tersedia
10	10_Evaluasi_Perbandingan	Tabel sandingan MAE/RMSE/MAPE/AIC/BIC kedua model + skor voting
11	11_Kesimpulan	Ringkasan akhir: model terbaik, order kedua model, RMSE, skor, scope & periode

Fungsi Pendukung	Deskripsi
<code>_style_header_row()</code>	Terapkan fill biru, font putih bold, alignment tengah, & border tipis pada baris header tabel
<code>_autosize()</code>	Sesuaikan lebar kolom otomatis berdasarkan panjang konten terpanjang (maks. 45)
<code>_write_df()</code>	Tulis DataFrame ke worksheet mulai dari baris tertentu, lalu styling header secara otomatis

Utilitas (app/utils)

decorators.py

PROTEKSI ROUTE

app/utils/decorators.py

Fungsi	Deskripsi
<code>login_required()</code>	Decorator yang membungkus view function admin. Memeriksa keberadaan <code>admin_id</code> pada Flask session; jika tidak ada, menampilkan flash message peringatan dan redirect ke halaman login sambil menyimpan tujuan asal (<code>?next=</code>) agar admin diarahkan kembali setelah berhasil login.

Skema Basis Data (Supabase / PostgreSQL)

Skema didefinisikan pada `sql/schema.sql` (dijalankan pertama) dan `sql/seed.sql` (akun admin default + master wilayah). Terdiri dari 12 tabel yang saling berelasi melalui foreign key.

Tabel	Deskripsi & Kolom Kunci
<code>admin_users</code>	Akun admin tunggal (autentikasi manual, bukan Supabase Auth). Kolom: id (uuid), username, password_hash (bcrypt), full_name, email, is_active, last_login_at
<code>wilayah</code>	Master kabupaten/kota. Kolom: id (serial), nama_wilayah (unique), kode_wilayah, is_top_kontributor
<code>data_historis</code>	Dataset training 2023–2025. Kolom: tanggal, wilayah_id (FK), nama_wilayah, total_berat, sumber_import. Unique (tanggal, nama_wilayah, sumber_import)
<code>data_aktual_2026</code>	Dataset pembandingan/uji 2026. Struktur identik dengan data_historis
<code>import_log</code>	Riwayat import file Excel: nama_file, tipe_dataset, jumlah_baris, status (sukses/gagal/sebagian), pesan, diupload_oleh (FK admin_users)
<code>forecast_run</code>	Tabel header tiap proses forecasting — menyimpan ± 40 kolom: scope wilayah, rentang data & prediksi, hasil ADF awal/diff, order & metrik ARIMA, order & metrik SARIMA, model_terbaik, skor_arima/sarima, status
<code>forecast_acf_pacf</code>	Nilai ACF/PACF per lag per run, ditandai tahap (sebelum_diff / sesudah_diff). FK run_id → forecast_run (cascade delete)
<code>forecast_dataset_point</code>	Snapshot data harian (train/test) yang dipakai pada satu run. FK run_id (cascade delete)
<code>forecast_prediction_test</code>	Prediksi per model (ARIMA/SARIMA) pada data test: nilai_aktual, nilai_prediksi, residual, batas_bawah/atas (CI 95%). FK run_id (cascade delete)
<code>forecast_prediction_future</code>	Hasil peramalan masa depan per model, termasuk nilai_aktual_2026 bila tanggalnya cocok. FK run_id (cascade delete)
<code>kontribusi_wilayah</code>	Ringkasan kontribusi penyaluran per kabupaten/kota: total_penyaluran, persentase, periode_mulai/akhir
<code>export_log</code>	Riwayat unduhan laporan Excel: run_id (FK), nama_file, diunduh_oleh (FK admin_users)

Seluruh tabel detail forecasting (`forecast_acf_pacf` , `forecast_dataset_point` , `forecast_prediction_test` , `forecast_prediction_future`) menggunakan `on delete cascade` terhadap `forecast_run` , sehingga menghapus satu run otomatis membersihkan seluruh data detail terkait tanpa baris yatim (orphan rows).

Trigger: fungsi `set_updated_at()` otomatis memperbarui kolom `updated_at` pada `admin_users` setiap kali baris diubah.

Lapisan Antarmuka (Frontend & Desain Sistem)

9.1 Struktur Template (Jinja2)

Berkas Template	Kegunaan
<code>base.html</code>	Layout induk — memuat Font Awesome & <code>style.css</code> via CDN, menyusun <code>app-shell</code> (sidebar + main-area), memuat Chart.js & <code>theme.js</code> di akhir body
<code>partials/sidebar.html</code>	Navigasi utama: brand mark, menu Dashboard, grup Data (Kelola Dataset, Import Excel, Data Historis, Data Aktual 2026, Kontribusi), grup Forecasting (Proses Baru, Riwayat), Profil, tombol theme switcher & logout
<code>partials/navbar.html</code>	Header halaman: breadcrumb, judul halaman, tombol hamburger (mobile)
<code>partials/flash.html</code>	Render flash message Flask (success/danger/warning/info) dengan auto-dismiss
<code>auth/login.html</code>	Form login admin
<code>auth/profile.html</code>	Form ubah profil & ubah password
<code>admin/dashboard.html</code>	Statistik ringkas, grafik tren penyaluran, top kontribusi wilayah, run forecasting terbaru
<code>admin/data_index.html</code>	Ringkasan kedua dataset & log import terbaru
<code>admin/data_upload.html</code>	Form upload Excel dengan preview sheet via AJAX
<code>admin/data_view.html</code>	Tabel & grafik data historis/aktual dengan filter wilayah & tanggal
<code>admin/data_kontribusi.html</code>	Tabel kontribusi penyaluran per kabupaten/kota
<code>admin/forecast_new.html</code>	Form parameter proses forecasting baru
<code>admin/forecast_index.html</code>	Riwayat seluruh run forecasting
<code>admin/forecast_detail.html</code>	Halaman paling kompleks — menampilkan seluruh tahapan pipeline: grafik train/test, korelogram ACF/PACF, parameter & evaluasi ARIMA/SARIMA, prediksi vs aktual, peramalan vs aktual 2026, tombol export Excel
<code>errors/404.html, 413.html, 500.html</code>	Halaman error kustom bertema sama dengan aplikasi

9.2 Sistem Desain Neumorphism

Didefinisikan pada `app/static/css/style.css` dengan token CSS variable yang konsisten:

Token	Nilai / Kegunaan
<code>--pink-500 / --sky-500 / --green-500</code>	Palet brand "Pinky · Sky Blue · Teeny Greeny" — dipakai pada gradient utama
<code>--gradient-brand</code>	linear-gradient 135° pink → sky → green, dipakai pada elemen aksentasi (header card, progress bar, toggle aktif)
<code>--nm-shadow-out / --nm-shadow-in</code>	Bayangan ganda (terang & gelap) khas neumorphism untuk efek timbul/tenggelam
<code>--bg / --surface / --ink-900...100</code>	Skala warna latar & teks, otomatis berganti nilai pada <code>data-theme="dark"</code>
<code>--font-display / --font-body</code>	Plus Jakarta Sans (judul/angka) dan Inter (teks badan)
<code>--radius-sm/md/lg/pill</code>	Skala radius sudut konsisten (12px–999px)

Mendukung **3 mode tema** (light / dark / system) yang disimpan di `localStorage` dengan key `lpg-theme`, diterapkan melalui atribut `data-theme` pada elemen `<html>`.

9.3 JavaScript & Interaktivitas

`app/static/js/theme.js`

- **Theme switcher** — fungsi `applyTheme()`, `initTheme()`, `setTheme()` mengatur & menyimpan preferensi tema.
- **Sidebar mobile** — toggle buka/tutup sidebar via hamburger button & backdrop overlay.
- **Auto-dismiss flash** — pesan notifikasi otomatis hilang dengan animasi fade setelah ± 5.2 detik.
- **Tabs** — sistem tab generik berbasis atribut `data-tabs` untuk mengelompokkan konten (mis. hasil ARIMA vs SARIMA pada halaman detail forecast).
- **Chart.js** — dimuat via CDN untuk seluruh grafik tren, korelogram, dan prediksi vs aktual.

Alur Kerja Sistem End-to-End

Tiga alur kerja utama yang dilakukan admin dari awal hingga laporan akhir.

Alur 1 — Import Dataset

- 1 **Upload File Excel**
Admin membuka `/data/upload`, memilih file & tipe dataset (historis / aktual_2026).
- 2 **Preview Sheet (AJAX)**
`/data/sheets` membaca nama-nama sheet pada file untuk dipilih admin sebelum import final.
- 3 **Validasi & Normalisasi**
`excel_import.process_upload()` memvalidasi kolom, menormalisasi tanggal/wilayah/angka, dan mengagregasi duplikat.
- 4 **Simpan ke Supabase**
`data_lpg.insert_batch()` meng-upsert data, otomatis mendaftarkan wilayah baru, lalu me-refresh tabel `kontribusi_wilayah`.
- 5 **Log & Notifikasi**
Hasil (sukses/gagal) dicatat ke `import_log` dan ditampilkan sebagai flash message ke admin.

Alur 2 — Proses Forecasting

- 1 **Isi Parameter**
Admin mengisi nama run, scope wilayah, rasio train/test, rentang tanggal prediksi, dan opsi bandingkan dengan data aktual 2026 di `/forecast/baru`.
- 2 **Eksekusi Pipeline**
`forecast_engine.run_full_pipeline()` dijalankan secara sinkron di server (10 tahap, lihat Bab VI.2).
- 3 **Penentuan Model Terbaik**
`determine_best_model()` membandingkan ARIMA vs SARIMA berdasarkan MAE/RMSE/MAPE pada data test.
- 4 **Persistensi Hasil**
Seluruh hasil (header run, ACF/PACF, dataset point, prediksi test, prediksi future) disimpan ke 5 tabel Supabase.
- 5 **Tampilkan Detail**
Admin diarahkan ke `/forecast/<run_id>` untuk melihat seluruh visualisasi & metrik secara lengkap.

Alur 3 — Export Laporan

- 1 **Klik Unduh Excel**
Dari halaman detail run, admin menekan tombol export menuju `/forecast/<run_id>/export`.
- 2 **Ambil Seluruh Data Run**
Server mengambil kembali header run beserta seluruh data detail dari Supabase.
- 3 **Susun Workbook 11-Sheet**
`excel_export.build_forecast_excel()` menyusun & menstilasi seluruh sheet laporan.
- 4 **Unduh & Catat Log**
File dikirim sebagai attachment ke browser admin; aktivitas dicatat ke `export_log`.

Keamanan & Catatan Produksi

- Sistem menggunakan **Supabase anon key** (bukan service role key) dengan RLS policy permisif (`USING (true)`) agar backend Flask dapat melakukan CRUD penuh — sesuai untuk skripsi/demo, namun **tidak direkomendasikan untuk produksi** dengan data sensitif publik.
- Untuk produksi sesungguhnya: gunakan **service role key** hanya di server (jangan diekspos ke client), perketat RLS per kebutuhan, dan terapkan rate-limiting pada endpoint upload.
- Session admin disimpan via Flask session dengan cookie `HttpOnly` & `SameSite=Lax` , masa berlaku 8 jam.
- Password admin di-hash menggunakan **bcrypt** (bukan disimpan plain-text).
- Akun admin default (`admin / admin123`) wajib segera diganti melalui menu Profil & Password setelah login pertama.

Troubleshooting

Masalah	Solusi
<code>ValueError: numpy.dtype size changed...</code> saat import <code>pmdarima</code>	Pastikan environment menggunakan <code>numpy<2</code> sesuai <code>requirements.txt</code> . Install ulang di virtualenv bersih.
Upload Excel gagal "Kolom tidak ditemukan"	Pastikan nama kolom persis: <code>Act. Gds Mvmnt Date</code> , <code>Kabupaten/Kota</code> , <code>Total Berat</code> .
Forecasting gagal "Tidak ada data historis"	Upload dataset historis terlebih dahulu melalui menu Import Excel.
Error koneksi Supabase	Periksa <code>SUPABASE_URL</code> & <code>SUPABASE_KEY</code> pada <code>.env</code> , serta pastikan <code>schema.sql</code> sudah dijalankan.

SiForLPG — Sistem Informasi Forecasting Permintaan LPG · Dokumentasi Teknis v1.0 · Juni 2026
Disusun berdasarkan analisis langsung terhadap source code proyek (app/, sql/, config.py, run.py)